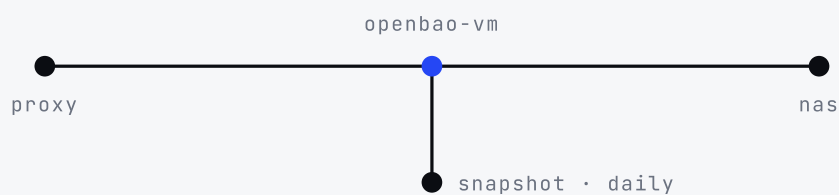


OPENBAO ON A SINGLE VM

A single node with Ansible and OpenTofu: installation, bootstrap, snapshots – and a disaster recovery you have rehearsed.



OpenBao on a VM

A single node with Ansible and OpenTofu: installation, bootstrap, snapshots – and a disaster recovery that has been rehearsed

Thomas Zachmann

17 September 2026

Contents

What this is about

- Why by hand, and why without AI
- A word about the values in this guide
- Who this is for
- The building blocks
- Two layers, cleanly separated
- The architecture in one picture
- Why one node and not three

Part I – What OpenBao does differently from Vault

Part II – Layer 1: Ansible

- What the role does
- The configuration
- The systemd hardening
- Deployment
- The NFS export

Part III – Layer 2: OpenTofu

- Why the provider is called `vault`
- What the configuration creates – and why
- The files
- Which value goes where
- The first run – with the root token
- Every subsequent run
- Rotating the password

Part IV – The bootstrap runbook

- Prerequisites
- 1. Log in with the root token
- 2. Apply the configuration with OpenTofu
- 3. Prove the replacement access – do not skip
- 4. Snapshot token – the one step that must stay manual
- 5. Revoke the root token
- 6. Final verification

Part V – Operation

- Unsealing after every restart
- Checking snapshots
- Verifying a snapshot by hand
- Restore into the same running instance
- Upgrade

Part VI – Disaster recovery: a brand new VM

What you must have

The part everybody gets wrong

Procedure

Restore drill

Can I actually restore? – checklist

Part VII – Deliberate decisions and known risks

Decisions

Risks

Troubleshooting

Appendix A – All commands

Appendix B – Glossary

What this is about

A secrets store has to do two things at once: be secure, and come back. Secure, so that nobody gets to secrets without authorisation. Come back, so that a dead disk, a broken upgrade or an accidental `rm -rf` is not the end of every credential ever stored in the company – or the homelab.

This guide builds OpenBao as a **single node on an Ubuntu VM**: Raft storage, Shamir unseal, TLS, systemd hardening, daily snapshots to a NAS. Ansible does the installation, OpenTofu does the configuration, and the few steps that have to stay manual are described as a runbook – in an order that rules out a lockout. At the end stands a disaster-recovery procedure onto an empty VM, including the question of how to rehearse it without endangering the running instance.

It is the third part of a series. N° 1 builds the same OpenBao in a Kubernetes cluster via Helm; N° 2 lets an application draw dynamic database credentials from it. The VM setup is the most mature of the three – many decisions in N° 1 are taken from here.

Why by hand, and why without AI

The automation in this guide – the Ansible role, the OpenTofu directory – is the result, not the path. Every line in it answers a question that was first asked by hand: why no `disable_mlock`? Why `MemorySwapMax=0`? Why is the root token gone after the bootstrap, and what happens if you revoke it too early?

Whoever asks an AI to “install OpenBao with Ansible” gets a role that runs. They do not get the insight that a `notify: restart openbao` takes the secrets store off the network, sealed, on the second playbook run. Those insights are the content of this guide; the role is merely their evidence.

The yardstick: whoever has worked through this guide can draw, unaided, which steps come after `init` in which order and which of them cannot be undone. And they can explain why a restore onto a new VM needs two different sets of unseal keys, one after the other.

A word about the values in this guide

The setup ran in a homelab. IP addresses, hostnames, the NAS type and the administrator’s username are replaced by placeholders:

Placeholder	Meaning
192.0.2.40	the OpenBao VM
192.0.2.41	the reverse proxy in front of it
192.0.2.160	the NAS with the NFS export <code>/volume1/openbao-backup</code>
bao.example.internal	the hostname under which OpenBao is reachable
admin	the user that replaces the root token

Unseal keys, root token, passwords and passphrases appear nowhere – they are not in any log in the original either, because the steps that produce them are deliberately never automated.

Who this is for

Readers who know Linux administration, systemd and the basics of Ansible and Terraform/OpenTofu, and have seen OpenBao or HashiCorp Vault before. The concepts – seal/unseal, policies, auth methods, audit – are assumed. If you want to learn them from the ground up, with labs that run on a laptop, that is in my book **Vault in Practice** (Leanpub, leanpub.com/vault-in-practice).

The building blocks

Component	Version	Role
Ubuntu Server	24.04 LTS	the VM, 1 vCPU / 2 GB / 20 GB
OpenBao	2.6.2, .deb from GitHub Releases	the secrets store
Ansible	2.17	layer 1: host, package, TLS, systemd, firewall, timers
OpenTofu	≥ 1.11	layer 2: audit, policies, auth methods
Reverse proxy	–	terminates TLS with a publicly valid certificate
NAS	–	NFS export as second location for snapshots

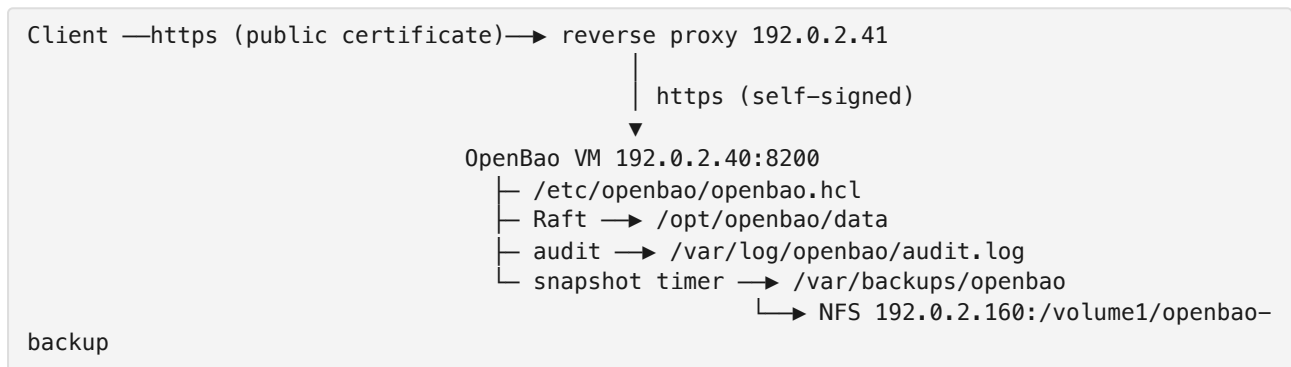
Two layers, cleanly separated

Layer	What	With	Needs an OpenBao token?
Host	package, TLS, systemd, firewall, snapshot timer	Ansible	no
Configuration	audit device, policies, auth methods	OpenTofu	yes

The dividing line is not “IaC versus CLI” but **configuration versus secret values**. The Ansible role stays deliberately token-free; everything that needs a token lives in OpenTofu. Secret *values* are in neither.

Only what has to stay manual stays manual: `init`, `unseal`, the root login for the very first `tofu apply`, and the snapshot token – a secret value that has to land as a file on the VM.

The architecture in one picture



Storage	integrated storage (Raft), one node
Unseal	Shamir 3 of 5, manual
TLS external	publicly valid certificate at the reverse proxy
TLS internal	self-signed, 10 years, generated by the role
Audit	file device, logrotate with SIGHUP reload
Backup	Raft snapshots via systemd timer, local + NFS

Why one node and not three

There is exactly **one** physical host. Three VMs on it do not protect against the dominant failure – host down – and with Shamir they triple the manual effort: 3 nodes × 3 keys = **9 entries per restart**. Protection here comes from backup and restore, not from quorum.

Growth stays possible: the certificate SANs already contain `bao-02` and `bao-03`, so a later `bao operator raft join` needs no reissue.

Part I – What OpenBao does differently from Vault

Everything below was verified against the actual `openbao_2.6.2_linux_amd64.deb` and the OpenBao documentation, not carried over from a Vault setup by search-and-replace.

	Vault	OpenBao 2.6.2
Installation	APT repo <code>apt.releases.hashicorp.com</code>	no APT repo – .deb from GitHub Releases, verified here against a signed <code>checksums.txt</code>
Package / binary	<code>vault / vault</code>	<code>openbao / bao</code>
Config	<code>/etc/vault.d/vault.hcl</code>	<code>/etc/openbao/openbao.hcl</code> (a package conffile)
Data / TLS	<code>/opt/vault/{data,tls}</code>	<code>/opt/openbao/{data,tls}</code>
Service / user	<code>vault.service / vault</code>	<code>openbao.service / openbao</code>
CLI variables	<code>VAULT_ADDR</code> , <code>VAULT_CACERT</code>	<code>BAO_ADDR</code> , <code>BAO_CACERT</code> (token helper still writes <code>~/.vault-token</code>)
<code>disable_mlock</code>	recommended <code>true</code> with Raft	gone – <code>mlock</code> removed, replaced by <code>MemorySwapMax=0</code>
Unauthenticated <code>sys/generate-root/*</code>	available	disabled by default since 2.5.3
<code>raft snapshot inspect</code>	exists	does not exist – only <code>save</code> and <code>restore</code>
Terraform provider	<code>hashicorp/vault</code>	still <code>hashicorp/vault</code> , pointed at OpenBao

The three that actually changed the setup:

No APT repository. The role downloads the .deb and builds its own trust chain: pinned GPG fingerprint → signed `checksums.txt` → SHA-256 of the package. Both links are mandatory; checking only the checksum would verify the file against a list an attacker could have swapped in the same request.

No mlock, therefore no swap argument. The packaged unit sets `MemorySwapMax=0`, so the process can never be swapped out even on a host that has swap. The role verifies beforehand that the host runs cgroup v2 (otherwise the limit is silently ignored) and asserts the *effective* value after every deployment – a future package revision cannot drop it unnoticed.

No snapshot inspect. The snapshot script verifies the archive itself: an OpenBao Raft snapshot is a gzipped tar holding `meta.json`, `state.bin` and a `SHA256SUMS` file in `coreutils` format over the two. The script recomputes both hashes – the same guarantee `inspect` gave, without scratch space.

Part II – Layer 1: Ansible

What the role does

roles/openbao/	
tasks/preflight.yml	OS, architecture, cgroup v2, swap, pending reboot, NFS export
tasks/install.yml	.deb download, GPG + SHA-256, dpkg hold
tasks/tls.yml	key, CSR, self-signed certificate, SAN assertion
tasks/config.yml	openbao.hcl, directories, logrotate
tasks/service.yml	systemd drop-in, MemorySwapMax assertion, status
tasks/firewall.yml	ufw (SSH first!)
tasks/snapshots.yml	NFS mount, timers, staleness check
tasks/helpers.yml	openbao-unseal, /etc/profile.d

preflight.yml aborts with instructions rather than deploying halfway: a pending reboot, wrong OS, cgroup v1, NFS export not shared – all of it is checked before the first apt.

The configuration

openbao.hcl replaces the conffile from the package. The shipped one uses storage "file" – a backend with no snapshot support. apt runs with the default force-confold, so a package upgrade keeps our file instead of opening an interactive prompt that would hang the play.

```

ui                = true
log_level         = "info"
log_format        = "json"

# The address CLIENTS use to reach OpenBao: the reverse proxy, not the VM.
# Otherwise redirects and the UI point nowhere.
api_addr          = "https://bao.example.internal"
cluster_addr      = "https://192.0.2.40:8201"

# No disable_mlock – on purpose. OpenBao removed mlock; the protection
# against secrets in swap is MemorySwapMax=0 in openbao.service.

storage "raft" {
  path      = "/opt/openbao/data"
  node_id   = "bao-01"
}

listener "tcp" {
  address            = "0.0.0.0:8200"
  cluster_address    = "0.0.0.0:8201"

  tls_cert_file     = "/opt/openbao/tls/openbao.crt"
  tls_key_file       = "/opt/openbao/tls/openbao.key"
  tls_min_version   = "tls12"

  # Default true since 2.5.3: an unauthenticated caller could otherwise
  # cancel any in-flight root generation. Break glass therefore depends on
  # the userpass admin – see the runbook.
  disable_unauthed_generate_root_endpoints = true

  # Without this EVERY audit entry shows the proxy IP instead of the client.
  # 127.0.0.1 does NOT belong here: x_forwarded_for_reject_not_present
  # defaults to true, and local init/unseal/snapshot calls send no XFF.
  x_forwarded_for_authorized_addrs = "192.0.2.41"
  x_forwarded_for_hop_skips        = "0"
}

# Deliberately NOT here: the audit device. It needs an API call and thus a
# token. The role stays token-free; audit comes from OpenTofu.

```

The systemd hardening

A drop-in on top of the unit from the package – deltas only, the base unit stays untouched and survives upgrades:

```

[Service]
# Replaces mlock. The base unit sets it too; re-asserted here so a future
# package revision cannot drop it unnoticed.
MemorySwapMax=0

# The base unit allows CAP_SYSLOG. We log JSON to stdout, journald takes it
# from there – no capability needed. An empty value resets the list.
AmbientCapabilities=
CapabilityBoundingSet=

# A core dump of OpenBao contains cleartext secrets from memory.
LimitCORE=0

# 'strict' additionally covers /opt and /var and requires the ReadWritePaths.
# The FIRST thing to revert if OpenBao fails to start after an upgrade:
# -e openbao_protect_system=full
ProtectSystem=strict
ReadWritePaths=/opt/openbao/data /var/log/openbao

ProtectKernelTunables=true
ProtectKernelModules=true
ProtectControlGroups=true
RestrictNamespaces=true
RestrictRealtime=true
RestrictSUIDSGID=true
LockPersonality=true
RestrictAddressFamilies=AF_INET AF_INET6 AF_UNIX

# Deliberately NOT set: MemoryDenyWriteExecute and SystemCallFilter.
# Stricter, but they can break plugins (separate processes). Hardening that
# breaks the service is worse than slightly less hardening.

```

Deployment

```
ansible-playbook playbooks/deploy-openbao.yml
```

As long as the VM is not yet in the NAS's NFS permission list, `snapshots.yml` deliberately aborts with instructions. Deploy without the NAS first, then add it:

```

# 1) Bring OpenBao into service, local snapshots active
ansible-playbook playbooks/deploy-openbao.yml -e openbao_snapshot_nfs_enabled=false

# 2) After adding the VM's IP to the NFS export:
ansible-playbook playbooks/deploy-openbao.yml

```

The second run needs **no restart** and therefore no re-unsealing – `openbao.hcl` does not change, only the mount unit and the timers are added.

Afterwards OpenBao is running and **sealed and uninitialised** – the correct state. `bao operator init` deliberately does not run through Ansible, so that unseal keys and root token never end up in task output, `-v` logs or terminal scrollbar.

The NFS export

The export is **shared** with an older Vault node. Two systems writing into one directory is only safe because the retention logic is anchored on the file name:

Node	Writes	Prunes
Vault	vault-<ts>.snap	vault-*.snap
OpenBao	openbao-<ts>.snap	openbao-*.snap

The prefix comes from `openbao_snapshot_prefix` and is used in exactly two places: the file name and the `find` pattern in `prune()`. **Changing one without the other means this node starts deleting the other machine's backups.** If that coupling ever feels too fragile, the answer is a second export, not a cleverer pattern.

On the NAS: squash “**map root to admin**” for the VM’s IP – the snapshot timer runs as root; without a matching mapping root is squashed and fails against the folder ACL. And: the push model means whoever compromises either VM can delete the snapshots on the NAS – **including the other machine’s**. Protection against that belongs on the NAS side: filesystem snapshots or a backup of the export. Without it the second copy only helps against disk failure, not against ransomware.

Part III – Layer 2: OpenTofu

Why the provider is called vault

There is no openbao/openbao provider in the registry. OpenBao kept the Vault HTTP API, so the HashiCorp provider works against it unchanged and is simply pointed at OpenBao via `VAULT_ADDR`. The risk: the projects will diverge. If a future provider release starts asserting Vault-specific version strings or Enterprise endpoints, pin the last working version in `versions.tf` rather than working around it in the resources.

What the configuration creates – and why

Resource	In OpenBao	Why
<code>vault_audit.file</code>	audit device writing to <code>/var/log/openbao/audit.log</code>	every access is recorded. Without it you can never answer who read a secret
<code>vault_policy.admin</code>	permission set	OpenBao denies everything by default
<code>vault_auth_backend.userpass</code>	login method	without an auth method only the root token gets in
<code>vault_userpass_auth_backend_user.admin</code>	user admin with the admin policy	your personal access
<code>vault_policy.snapshot</code>	read on <code>sys/storage/raft/snapshot</code> only	the backup timer needs a token that may do exactly one thing

The purpose in one sentence: so that the root token can be revoked. It is valid indefinitely, never expires and cannot be scoped.

Under OpenBao that sentence carries more weight than under Vault. The `admin` policy contains `sys/generate-root-token/*`, because OpenBao disables the unauthenticated `sys/generate-root/*` endpoints since 2.5.3. Three unseal keys alone no longer mint a root token – this login is the break-glass path and has to work before root goes away.

The files

```
# versions.tf
terraform {
  required_version = ">= 1.11.0"    # write-only attributes from 1.11

  required_providers {
    vault = { source = "hashicorp/vault", version = "~> 5.0" }
  }

  # State and plan are encrypted client side: the repo lives in a cloud
  # sync, and an unencrypted state with sensitive values would go to a
  # third party, versioned.
  encryption {
    key_provider "pbkdf2" "main" { passphrase = var.state_passphrase }
    method "aes_gcm" "main"      { keys = key_provider.pbkdf2.main }
    state { method = method.aes_gcm.main }
    plan  { method = method.aes_gcm.main }
  }
}

provider "vault" {
  # Deliberately EMPTY. Address from VAULT_ADDR, token from ~/.vault-token.
  # A token in HCL ends up in the configuration AND the state.
}
```

```
# audit.tf - must exist FIRST so that everything further is recorded.
# Enforced not by a manual step but by depends_on on the other resources.
resource "vault_audit" "file" {
  type = "file"
  path = "file"
  options = { file_path = var.audit_log_path }
}
```

```
# auth-userpass.tf
resource "vault_auth_backend" "userpass" {
  type      = "userpass"
  path      = "userpass"
  depends_on = [vault_audit.file]
}

resource "vault_userpass_auth_backend_user" "admin" {
  mount      = vault_auth_backend.userpass.path
  username   = var.admin_username

  # password_wo is a WRITE-ONLY attribute: the value goes to OpenBao but
  # NEVER into the state or a plan file. Price: no drift detection.
  # Rotation goes through bumping password_wo_version.
  password_wo      = var.admin_password
  password_wo_version = var.admin_password_version

  token_policies = [vault_policy.admin.name]
  token_ttl      = 3600
  token_max_ttl  = 28800
}
```

```
# variables.tf (excerpt)
variable "admin_password" {
  type      = string
  ephemeral = true    # used during the run, persisted nowhere
  # NO default: the provider insists on exactly one of password_wo /
  # password_hash_wo – with null even the plan fails. So the variable has
  # to be set on EVERY run, even when only a policy changes.
}
```

The admin policy in full – deliberately broad, this login drives OpenTofu:

```
path "sys/health"           { capabilities = ["read", "sudo"] }
path "sys/capabilities-self" { capabilities = ["update"] }
path "sys/mounts"           { capabilities = ["read", "list"] }
path "sys/mounts/*"         { capabilities = ["create", "read", "update", "delete", "list", "sudo"] }
path "sys/auth"             { capabilities = ["read", "list"] }
path "sys/auth/*"           { capabilities = ["create", "read", "update", "delete", "sudo"] }
path "sys/policies/acl"     { capabilities = ["list"] }
path "sys/policies/acl/*"   { capabilities = ["create", "read", "update", "delete", "list"] }
path "sys/audit"            { capabilities = ["read", "list", "sudo"] }
path "sys/audit/*"          { capabilities = ["create", "read", "update", "delete", "sudo"] }
path "sys/leases/*"         { capabilities = ["create", "read", "update", "delete", "list", "sudo"] }
path "sys/storage/raft/snapshot" { capabilities = ["read"] }

# Break glass – specific to OpenBao. Without these paths this login cannot
# mint a replacement root token, and the unseal keys alone cannot either.
path "sys/generate-root-token/attempt" { capabilities = ["create", "read", "update", "delete", "sudo"] }
path "sys/generate-root-token/update"   { capabilities = ["create", "update", "sudo"] }
path "sys/decode-token"                 { capabilities = ["create", "update"] }

# auth/token/create is mandatory: the provider creates a short-lived child
# token per run and needs update on it.
path "auth/*"           { capabilities = ["create", "read", "update", "delete", "list", "sudo"] }
path "identity/*"       { capabilities = ["create", "read", "update", "delete", "list"] }
path "secret/*"         { capabilities = ["create", "read", "update", "delete", "list"] }
path "kubernetes/*"     { capabilities = ["create", "read", "update", "delete", "list"] }
path "pki/*"            { capabilities = ["create", "read", "update", "delete", "list", "sudo"] }
```

Which value goes where

Both passwords are **yours to invent**. They are not fetched from anywhere – they come into existence the moment you set them for the first time.

Variable	What it is	What it is not
TF_VAR_state_passphrase	passphrase with which OpenTofu encrypts its state . At least 16 characters. Purely local, OpenBao never sees it	not an unseal key, not the root token
TF_VAR_admin_password	your personal login password for OpenBao	not your Linux or SSH password
VAULT_ADDR	the address, read by the provider	–
BAO_ADDR	the same address, read by the bao CLI	–

Setting only one of the two is the classic “works in tofu, fails in the shell” trap. An `env.sh` sets both together and loads the passwords from the operating system’s keychain – without ever displaying them:

```
export VAULT_ADDR="https://bao.example.internal"
export BAO_ADDR="$VAULT_ADDR"
read -rs -p "State passphrase: " TF_VAR_state_passphrase; echo
read -rs -p "OpenBao admin password: " TF_VAR_admin_password; echo
export TF_VAR_state_passphrase TF_VAR_admin_password
```

`read -rs` echoes nothing and leaves no trace in the history – unlike `export VAR='secret'`, which stays in `~/.zsh_history` permanently.

Why random rather than memorable: `openssl rand -base64 32` is 256 bits of entropy in 44 characters. PBKDF2 is vulnerable precisely when the passphrase itself is weak – key derivation only slows brute force by a factor. With 256 bits, brute force is hopeless regardless of the iteration count. You never have to type it – it comes from the keychain.

The first run – with the root token

Chicken and egg: the access this configuration creates does not exist yet. Exactly **once** with the root token:

```
cd tofu
source env.sh

# Read the root token without echo instead of exporting it into the history.
# VAULT_TOKEN, not BAO_TOKEN – this one is read by the provider.
read -rs -p "Root token: " VAULT_TOKEN; echo; export VAULT_TOKEN

tofu init
tofu plan      # expected: 5 to add, 0 to change, 0 to destroy
tofu apply

unset VAULT_TOKEN # no longer needed from here on
```

Every subsequent run

```
source env.sh
bao login -method=userpass username=admin # token lands in ~/.vault-token
tofu plan
```

VAULT_TOKEN is deliberately **not** set: the provider finds the token from `bao login` in `~/.vault-token` by itself. A leftover VAULT_TOKEN would silently override it.

The provider creates a **short-lived child token per run** and revokes it afterwards. That is why additional token events appear in the audit log for every `tofu plan` – expected behaviour, not a fault.

Rotating the password

Write-only attributes have no drift detection. The change has to be signalled through the counter:

```
export TF_VAR_admin_password='new-password'
tofu apply -var admin_password_version=2
```

Then record the new value permanently in `admin_password_version`.

Part IV – The bootstrap runbook

One-off sequence after `bao operator init`. **The order is not arbitrary.** The audit device comes first so that every following step is recorded. The root token is revoked last, only after the replacement access has been *proven*.

Why step 3 matters more under OpenBao than under Vault. OpenBao disables the unauthenticated `sys/generate-root/*` endpoints, and `bao operator generate-root` drives the authenticated `sys/generate-root-token` endpoints instead. Holding all five unseal keys is therefore **no way back** on its own – you also need a login with `sys/generate-root-token/*`, which the `admin` policy from step 2 provides. Under Vault, skipping step 3 was forgivable. Here it is not. **Do not run step 5 until step 3 has passed.**

Prerequisites

```
ssh ubuntu@192.0.2.40
bao status           # Initialized true, Sealed false
```

If it says `Sealed true`: `sudo openbao-unseal`. A `/etc/profile.d` snippet sets `BA0_ADDR` and `BA0_CACERT` for interactive shells, so plain `bao` commands work on the VM.

1. Log in with the root token

Exactly **once**. The access that replaces root does not exist yet.

```
bao login
bao token lookup -format=json | jq -r '.data.policies' # ["root"]
```

2. Apply the configuration with OpenTofu

Part III, “The first run”. Then verify:

```
tofu output
bao audit list -detailed
bao policy list      # admin, snapshot, default, root
bao auth list        # userpass/ and token/
```

3. Prove the replacement access – do not skip

```
# -token-only is critical (shorthand for -field=token -no-store).
# WITHOUT this flag 'bao login' overwrites ~/.vault-token and thereby ends
# your running root session - BEFORE it is proven that the new access
# works. That is how a verification turns into a lockout.
ADMIN_TOKEN="$(bao login -method=userpass -token-only username=admin)"

BAO_TOKEN="$ADMIN_TOKEN" bao token lookup -format=json | jq -r '.data.policies, .data.ttl'
BAO_TOKEN="$ADMIN_TOKEN" bao policy list

# The break-glass path specifically - what unseal keys alone can no longer
# do. -init starts a root generation, -cancel aborts it: proves the
# capability without producing a root token you would then have to handle.
BAO_TOKEN="$ADMIN_TOKEN" bao operator generate-root -init
BAO_TOKEN="$ADMIN_TOKEN" bao operator generate-root -cancel

unset ADMIN_TOKEN
```

Only once admin appears, `bao policy list` works **and** the `generate-root -init/-cancel` pair succeeds is the replacement access proven. **If any of it fails, do not run step 5 under any circumstances.**

4. Snapshot token – the one step that must stay manual

The policy comes from OpenTofu. The **token** does not: it is a secret *value* that has to land as a file on the VM. `vault_token` would write it into the state.

```
# -period: a PERIODIC token. A regular one expires and the snapshots stop
# silently - the single most common backup failure mode. Every use extends
# it, so the daily timer keeps it alive by itself.
bao token create \
  -policy=snapshot \
  -period=720h \
  -orphan \
  -display-name=openbao-snapshot \
  -field=token \
  | sudo tee /etc/openbao/snapshot.token >/dev/null

sudo chmod 0600 /etc/openbao/snapshot.token
```

`-orphan` requires `sudo` capability in OpenBao – only possible while you are still root. Verify:

```
sudo systemctl start openbao-snapshot.service
sudo journalctl -u openbao-snapshot.service -n 10 --no-pager -o cat
sudo sh -c 'ls -lh /var/backups/openbao/'
```

Expected: Snapshot created: `/var/backups/openbao/openbao-<ts>.snap`. Without the NAS mount the job ends with an error – deliberately: a backup that never reached the second location must not count as success.

5. Revoke the root token

Only if step 3 succeeded – including the generate-root check.

```
bao token lookup -format=json | jq -r '.data.policies' # ["root"]
bao token revoke -self
bao token lookup # must now fail
```

From here on userpass is the way in. Break glass if you lose that access:

```
bao login -method=userpass username=admin
bao operator generate-root -init
bao operator generate-root # 3x with unseal keys
```

If both the root token *and* the admin login are gone, the last resort is to re-enable the legacy path deliberately and temporarily:

```
ansible-playbook playbooks/deploy-openbao.yml \
  -e openbao_disable_unauthed_generate_root_endpoints=false \
  -e openbao_allow_restart=true
ssh ubuntu@192.0.2.40 sudo openbao-unseal
bao operator generate-root -init # now works without a token
```

Turn it back off in the same session. Leaving it on is exactly the exposure the default is there to prevent.

6. Final verification

First: did the snapshot token survive the root revocation? The most important and most easily forgotten check. Tokens **inherit**: revoking a parent takes its children with it. That is exactly why the snapshot token was created with `-orphan`.

```
sudo systemctl start openbao-snapshot.service # must exit 0
sudo sh -c 'BA0_TOKEN=$(cat /etc/openbao/snapshot.token) bao token lookup -format=json' \
| jq -r '.data.orphan, .data.policies, .data.period'
```

Expected: true, ["default","snapshot"], 2592000. default belongs there – the token needs it to renew itself. If orphan is **false**, the token died with root. The way back is generate-root via the admin login, because `-orphan` requires elevated permissions.

Then:

```
bao login -method=userpass username=admin # here deliberately WITH storing
bao status | grep -E 'Sealed|Initialized' # false / true
bao audit list # file/
bao auth list # userpass/ and token/
bao policy list # admin, snapshot, default, root
systemctl list-timers 'openbao-*' # both timers active
sudo sh -c 'ls -lh /var/backups/openbao/' # at least one snapshot
sudo sh -c 'ls -lh /mnt/snapshots/' # openbao-*.snap next to vault-*.snap
```

Read that last line carefully: the export is shared with the Vault node. You should see **both** prefixes, and the count of `vault-*.snap` must not have changed.

Part V – Operation

Unsealing after every restart

```
sudo openbao-unseal
```

A small script that reads `bao status`, shows progress and calls `bao operator unseal` without an argument – keys are read interactively and without echo, never passed as arguments (visible via `/proc`, stored in the history). This is the deliberately accepted price of the Shamir decision. As long as nothing depends on it, it is merely inconvenient. Once a cluster with External Secrets hangs off it, every power cut becomes a cluster outage – at that point reconsider auto-unseal (transit or cloud KMS).

Checking snapshots

```
systemctl list-timers 'openbao-*'  
sudo sh -c 'ls -lh /var/backups/openbao/ /mnt/snapshots/'  
systemctl status openbao-snapshot-check.service
```

`openbao-snapshot-check.timer` warns daily if the last **successful** run is older than 48 h. It deliberately checks success rather than the timer – the most common backup failure mode is an expired token where the timer keeps firing and every run fails.

Verifying a snapshot by hand

OpenBao has no `bao operator raft snapshot inspect`. The archive is a gzipped tar, so this works anywhere:

```
snap=/mnt/snapshots/openbao-<ts>.snap  
  
tar -tzf "$snap"                # meta.json, state.bin, SHA256SUMS  
tar -xz0f "$snap" meta.json | jq .    # index, term, version, size  
tar -xz0f "$snap" SHA256SUMS        # expected hashes  
tar -xz0f "$snap" state.bin | sha256sum  # must match the line above
```

That last pair is exactly what `openbao-snapshot` checks after every save, before the file is moved into place.

Restore into the same running instance

Data deleted by accident, OpenBao still running – the unseal keys match, so **without** `-force`:

```
bao operator raft snapshot restore /mnt/snapshots/openbao-<ts>.snap  
sudo openbao-unseal                # sealed afterwards
```

Verify the file first with the tar commands above – there is no `inspect` to do it for you.

Upgrade

```
ansible-playbook playbooks/deploy-openbao.yml \
  -e openbao_version=2.6.3 -e openbao_allow_restart=true
ssh ubuntu@192.0.2.40 sudo openbao-unseal
```

The package is held via `dpkg hold`; the role releases the hold, downloads and verifies the new `.deb`, installs it and re-applies the hold. Record the new version in **both** places – inventory and task runner – otherwise the next plain `deploy` pins the server back.

Part VI – Disaster recovery: a brand new VM

For the case where the VM no longer exists: dead disk, deleted VM, corrupted Raft data.

This procedure is unverified until you have run through it once. The section “Restore drill” explains how to do that safely. **Right now** is the best time: as long as nothing depends on this instance, a drill is low risk. Once a cluster consumes secrets from it, it becomes considerably more delicate.

What you must have

	Where it lives	Without it ...
Unseal keys of the ORIGINAL cluster (3 of 5)	password manager	the snapshot is worthless – it is encrypted with them
A snapshot <code>openbao-*.snap</code>	NAS	there is nothing to restore
The repository	Git	you rebuild the VM by hand
SSH key	workstation	no access to the new VM
The admin password	password manager	see below

Keys and snapshots must not live in the same place. If the unseal keys sit on the same NAS as the snapshots, a single loss takes both.

One OpenBao-specific addition to that list. Under Vault, three unseal keys were enough to mint a new root token via the unauthenticated endpoints. OpenBao disables those. After a restore the userpass admin from the snapshot is back – so **the password for admin is now part of your recovery material**, not just a convenience. If it is lost, the only way in is to re-enable the legacy endpoints deliberately (runbook, step 5).

Pick the right file. `vault-*.snap` files are the other machine’s backups and are useless here – they are encrypted with a different set of unseal keys. Only `openbao-*.snap` belongs to this system.

The part everybody gets wrong

Restoring onto a **new** cluster involves **two different sets** of unseal keys, one after the other:

1. New VM: `'bao operator init'` → generates NEW keys + a NEW root token
2. Unseal with the NEW keys → OpenBao is operational but empty
3. Restore the snapshot (`-force`) → overwrites the keyring from the snapshot
4. From now on the ORIGINAL keys → the new ones are worthless

`-force` is required because the Shamir keys of the fresh cluster are not consistent with the snapshot data, which came from a different cluster. After the restore, unseal with the **original** keys until the original threshold is reached.

Consequence: the keys and root token generated in step 1 are throwaway material – you need them only for the minutes between step 2 and step 3. Afterwards everything from the original applies again: root token, policies, the admin user with the original password, auth methods.

Procedure

1. Provision a new VM. Ubuntu 24.04, 1 vCPU / 2 GB / 20 GB. Reuse the IP if at all possible – otherwise adjust the NFS permission, the proxy upstream and the inventory. The host key will change: `ssh-keygen -R 192.0.2.40`.

2. Run the Ansible role.

```
ansible-playbook playbooks/deploy-openbao.yml
```

Result: OpenBao running, **uninitialised and sealed**. Exactly right. The NFS mount comes with it – that is how you fetch the snapshot in the next step. `openbao-snapshot.service` will fail for now (“No snapshot token”) – expected, fixed in step 7.

3. Stage the snapshot. It has to be a **local file**:

```
ssh ubuntu@192.0.2.40
sudo sh -c 'ls -lt /mnt/snapshots/openbao-*.snap | head'
sudo cp /mnt/snapshots/openbao-<ts>.snap /tmp/restore.snap

sudo tar -tzf /tmp/restore.snap
sudo tar -xz0f /tmp/restore.snap meta.json | jq .
sudo sh -c 'tar -xz0f /tmp/restore.snap SHA256SUMS'
sudo sh -c 'tar -xz0f /tmp/restore.snap state.bin | sha256sum'
```

The hash from the last command must appear in the SHA256SUMS output next to `state.bin`. If it does not – or if tar errors out – take the next older file. This is the moment that decides whether the 30 copies on the NAS were worth anything.

4. Initialise and unseal temporarily. These keys are **throwaway**.

```
sudo -i
export BAO_ADDR=https://127.0.0.1:8200

bao operator init -key-shares=1 -key-threshold=1    # 1 is enough, they are temporary
bao operator unseal <temp-key>
export BAO_TOKEN=<temp-root-token>
bao status          # Sealed false
```

5. Restore the snapshot.

```
bao operator raft snapshot restore -force /tmp/restore.snap
```

`-force` is **mandatory**: this cluster’s Shamir keys do not match the data in the snapshot, and without the flag OpenBao refuses for exactly that reason. It seals itself afterwards because the keyring was replaced.

6. Unseal with the ORIGINAL keys.

```
bao operator unseal      # 3x, with the keys of the ORIGINAL cluster
bao status               # Sealed false, Total Shares 5, Threshold 3
```

From here everything is back: policies, audit configuration, userpass, the admin user with the original password, the original root token.

7. Follow-up work. Create a new snapshot token – the old one exists again inside the data, but its *value* only ever lived in the file on the old VM:

```
bao login -method=userpass username=admin
bao token create -policy=snapshot -period=720h -orphan \
  -display-name=openbao-snapshot -field=token \
  | sudo tee /etc/openbao/snapshot.token >/dev/null
sudo chmod 0600 /etc/openbao/snapshot.token
sudo systemctl start openbao-snapshot.service
sudo rm -f /tmp/restore.snap
```

Reconcile OpenTofu – the state describes the situation before the outage:

```
cd tofu && source env.sh
bao login -method=userpass username=admin
tofu plan      # expected: "No changes"
```

If plan reports changes, the snapshot predates your last configuration change. Then `tofu apply` – which is precisely why the configuration is kept as code.

8. Verify.

```
bao status               # Initialized true, Sealed false, Version 2.6.2
bao audit list           # file/
bao auth list            # userpass/
bao policy list          # admin, snapshot, default, root
bao login -method=userpass username=admin # the original password must work
systemctl list-timers 'openbao-*'
sudo sh -c 'ls -lh /var/backups/openbao/ /mnt/snapshots/'
curl -s -o /dev/null -w '%{http_code}\n' https://bao.example.internal/v1/sys/health # 200
```

Check the NAS listing for collateral damage as well: the count of `vault-*.snap` must be unchanged. A restored node running with a wrong `openbao_snapshot_prefix` would prune the other machine's backups on its first timer run.

Restore drill

Do not practise on the production VM. Set up a second VM and walk through the procedure above – with one additional rule:

The drill VM must be network isolated. A restored instance holds every lease of the original. If it runs in parallel and is reachable, it can revoke third-party credentials and thereby invalidate leases of your real instance.

In practice: boot the drill VM in an isolated network or without a NIC, access only through the console; transfer the snapshot as a file; leave the proxy and firewall alone; deploy with `-e openbao_snapshot_nfs_enabled=false` – a drill VM must **never** mount the shared NFS export, its snapshot timer would prune `openbao-*.snap` there against *its* retention.

Can I actually restore? – checklist

Answerable without an actual outage:

- ☐ Are the 5 unseal keys in the password manager, **not** on the NAS?
- ☐ Is the password for admin in the password manager? (Under OpenBao it is recovery material.)
- ☐ Is the newest `openbao-*.snap` on the NAS less than 24 h old?
- ☐ Is it readable? `tar -tzf` plus the SHA256SUMS check from step 3
- ☐ Does the NAS survive the loss of the VM? (yes – separate device)
- ☐ Does it survive an attacker encrypting the VM? **Currently no** – the VM may delete files on the NAS. Filesystem snapshots on the NAS would close that gap.
- ☐ Have I walked through the procedure **at least once**?

Part VII – Deliberate decisions and known risks

Decisions

No `disable_mlock` – OpenBao removed `mlock` entirely, the option is obsolete. Secrets stay out of swap via `MemorySwapMax=0`; preflight checks `cgroup v2`, `service.yml` asserts the effective value after every run.

.deb with its own trust chain – no APT repo, so no repository signature apt could check. Pinned fingerprint → signed `checksums.txt` → SHA-256. Downloading only when the version actually changes.

`disable_unauthed_generate_root_endpoints` left at `true` – the secure default. The cost is that break glass moves into the bootstrap order: prove the userpass admin *first*, then revoke root.

No `notify: restart openbao` – a restart **seals**. A naive handler would take OpenBao off the network on the second playbook run and leave it sealed. Enable deliberately: `-e openbao_allow_restart=true`.

`x_forwarded_for_authorized_addrs` without `127.0.0.1` – `x_forwarded_for_reject_not_present` defaults to `true`. With `localhost` in the list, local `init/unseal` calls (which send no XFF header) would be rejected.

SANs including `localhost` and `127.0.0.1` – otherwise every local bao call needs `-tls-skip-verify`. The wrong habit when unsealing a secrets store.

`systemd timer` instead of built-in snapshot automation – `sys/storage/raft/snapshot-auto` is a Vault Enterprise feature that OpenBao does not have either.

`MemoryDenyWriteExecute` and `SystemCallFilter` not set – stricter, but they can break plugins. Hardening that breaks the service is worse than slightly less hardening.

Risks

Risk	Status
Sealed after every reboot until 3 keys are entered	deliberately accepted
Unseal keys alone are not a way back in – break glass needs a working login	mitigated by runbook ordering + sys/generate-root-token/* in the admin policy
Shared NFS export: a wrong prune pattern would delete the Vault node's backups	prefix-anchored find; documented in three places
Push backup: whoever compromises a VM can delete the NAS snapshots of both systems	defence belongs on the NAS side
Write-back cache without a healthy controller battery ⇒ Raft/BoltDB corruption on power loss	check the controller, clarify UPS
Single node, single host	protection through restore, not quorum
The hashicorp/vault provider targets Vault, not OpenBao	works today; pin the last known good version if a release starts asserting Vault specifics
Restore never tested	see Part VI

Troubleshooting

Symptom	Cause	Fix
Proxy returns 400 , body Client sent an HTTP request to an HTTPS server.	upstream is set to http, the listener speaks TLS	upstream to https://192.0.2.40:8200
Proxy returns 502	OpenBao is not running or unreachable	systemctl status openbao, check the ufw rule for 192.0.2.41
400 with an empty body from OpenBao	XFF header missing, x_forwarded_for_reject_not_present is true	the proxy must set X-Forwarded-For
501	reachable but not initialised	expected before bao operator init
Error loading CA File: permission denied	BA0_CACERT points into /opt/openbao/tls (0750 root:openbao)	use the world-readable copy under /usr/local/share/ca-certificates/
Snapshot service failed , Copy to /mnt/snapshots failed	NAS directory shows mode 000, root is squashed	NFS permissions: squash "map root to admin" for the VM's IP
Snapshot service failed but the local snapshot exists	Intended. A backup that did not reach the second location does not count	fix NAS access
Snapshot failed its integrity check	archive did not decompress or hashes do not match	the file was discarded on purpose; check disk and journal, run the service again
ls: cannot access '/var/backups/openbao/*.snap' despite sudo	the glob is expanded before sudo, as the normal user without read access	sudo sh -c 'ls /var/backups/openbao/'
Service fails to start after an upgrade	ProtectSystem=strict blocking a path the new version writes	-e openbao_protect_system=full, then find the path
tofu: Failed to request input for var.state_passphrase	TF_VAR_state_passphrase not set; the encryption block is evaluated statically, a prompt is impossible	source env.sh
tofu apply: 403 permission denied mid-run	token expired (TTL 1 h) – or ~/.vault-token belongs to the other instance	bao token lookup, then bao token renew or log in again

Appendix A – All commands

```
# — Layer 1: Ansible —————
ansible-playbook playbooks/deploy-openbao.yml -e openbao_snapshot_nfs_enabled=false
ansible-playbook playbooks/deploy-openbao.yml # after the NFS export is shared

# — init / unseal (on the VM, normal terminal) —————
export BAO_ADDR=https://127.0.0.1:8200
bao operator init -key-shares=5 -key-threshold=3 # output -> password manager
sudo openbao-unseal

# — Layer 2: OpenTofu, first run —————
bao login # root token, once
cd tofu && source env.sh
read -rs -p "Root token: " VAULT_TOKEN; echo; export VAULT_TOKEN
tofu init && tofu plan && tofu apply
unset VAULT_TOKEN

# — Prove the replacement access —————
ADMIN_TOKEN="$(bao login -method=userpass -token-only username=admin)"
BAO_TOKEN="$ADMIN_TOKEN" bao policy list
BAO_TOKEN="$ADMIN_TOKEN" bao operator generate-root -init
BAO_TOKEN="$ADMIN_TOKEN" bao operator generate-root -cancel
unset ADMIN_TOKEN

# — Snapshot token —————
bao token create -policy=snapshot -period=720h -orphan \
  -display-name=openbao-snapshot -field=token | sudo tee /etc/openbao/snapshot.token >/dev/
  null
sudo chmod 0600 /etc/openbao/snapshot.token
sudo systemctl start openbao-snapshot.service

# — Revoke root (only after the test passed) —————
bao token revoke -self

# — Operation —————
sudo openbao-unseal
systemctl list-timers 'openbao-*'
tar -xz0f /mnt/snapshots/openbao-<ts>.snap state.bin | sha256sum
bao operator raft snapshot restore /mnt/snapshots/openbao-<ts>.snap # same instance
bao operator raft snapshot restore -force /tmp/restore.snap # new VM
```

Appendix B – Glossary

Shamir 5/3 – The master key is split into five shares, any three reconstruct it. After every restart three must be entered.

Raft – Integrated storage backend. Sensible even with one node, because only Raft can snapshot save/restore.

Root token – From `init`. Valid indefinitely, cannot be scoped. Revoked after the bootstrap.

Break glass – The route to a new root token. Under OpenBao: unseal keys **and** a login with `sys/generate-root-token/*`.

Periodic token – A token with `-period` that extends itself by that span on every use. For timer jobs that must never expire.

Orphan token – A token without a parent. Survives the revocation of the token that created it.

Write-only attribute (`password_wo`) – A Terraform/OpenTofu attribute that goes to the provider but is never written to state or plan.

Conffile – A configuration file shipped by a Debian package and treated specially on upgrades. `force-confold` keeps the local version.

MemorySwapMax=0 – cgroup v2 limit that forbids a process to swap. Replaces the removed `mlock` under OpenBao.

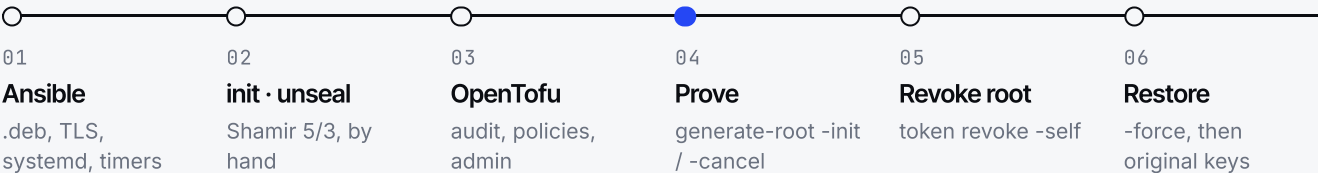
XFF – X-Forwarded-For, the header a reverse proxy uses to pass on the client IP. Without configuration the audit log only shows the proxy.

A secrets store has to do two things at once: be secure, and come back.

This guide builds OpenBao as a single node on a VM – Raft, Shamir, TLS, systemd hardening, daily snapshots to a NAS. Ansible installs, OpenTofu configures, and the few steps that have to stay manual are laid out as a runbook.

With what OpenBao does differently from Vault: no APT repo, no mlock, no snapshot inspect, no unauthenticated generate-root endpoints. And a disaster recovery onto an empty VM that explains why it takes two sets of unseal keys, one after the other.

```
-- The order that rules out a lockout
tofu apply                                # audit, policies, admin
bao operator generate-root -init          # prove break glass
bao operator generate-root -cancel
bao token revoke -self                   # only now
```



ubuntu	24.04 LTS · 1 vCPU · 2 GB
openbao	2.6.2 · .deb · raft
ansible	2.17 · layer 1
opentofu	≥ 1.11 · layer 2 · encrypted state
backup	systemd timer · NFS

33 pages · 7 parts · 2 appendices
Deep dive: **Vault in Practice**
leanpub.com/vault-in-practice
Set in Inter Tight, Inter and JetBrains Mono.
All output is real terminal output.